

Module 4: Supervised Learning
Submodule: 4.1 Linear Regression
Lecture: Linear Regression

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I	Introduction	1
	Examples of Input-Output Pairs	2
II	Training Set	2
III	Finding the Hypothesis Function h	2
IV	Generalization to New Data	3
V	Classification vs. Regression	3
VI	Example Data: House Prices	3
VII	Representation of h	4
	Affine (Linear) Functions	5
	Training Set and Data Representation	5
	Mathematical Representation of Hypothesis h	6
VIII	Cost function	8
IX	Common questions	8
X	Summary	8

I. Introduction

Supervised learning is one of the most common approaches in machine learning, where a model is trained on a dataset containing **input-output**

pairs. This section introduces the concept of "supervised" learning and discusses its fundamental aspects.

In this framework, we aim to learn a **prediction** function h that maps **inputs** X to **outputs** Y , denoted as:

$$h : X \longrightarrow Y$$

We then use h on *new* data.

Examples of Input-Output Pairs

Several examples of supervised learning tasks include:

- **Image Classification:** Given an image x , determine whether it contains a cat y .
- **Text Classification:** Given some text x , determine if it contains hate speech y .
- **Structured Data:** Given some structured data x , such as house attributes, predict the price of the house y .

II. Training Set

In supervised learning, we are given a **training set**, which consists of input-output pairs:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\} \quad (1)$$

where:

- $x^{(i)} \in X$: is an element from the input space X .
- $y^{(i)} \in Y$: is its corresponding label from the output space Y .

III. Finding the Hypothesis Function h

The goal is to find a good h , often called the **hypothesis function**. A good $h : X \longrightarrow Y$ should generalize well to new data.

- The function h is determined using a **training algorithm**.
- Algorithms like **Stochastic Gradient Descent (SGD)** are commonly used.

IV. Generalization to New Data

It is crucial to assess h 's ability to generalize to new, unseen data:

- Use a **test set** or a **development set**, which is used to evaluate the model's performance on new data.
- The training process is not just about memorizing the training data but about learning patterns that can be applied to future data.
- As an example, a model trained on a specific set of images should be able to classify newly introduced images correctly.

This is called “prediction.” We are *very* interested in x that is not an element of the training set ($x \notin \text{trainingset}$).

V. Classification vs. Regression

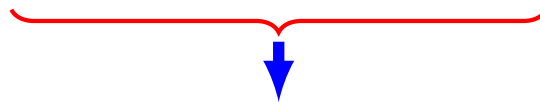
- If Y is **discrete**, the process is called **classification**.
 - Example: Determining whether a text contains hate speech or not.
- If Y is **continuous**, the process is called **regression**.
 - Example: Predicting house prices based on features.

VI. Example Data: House Prices

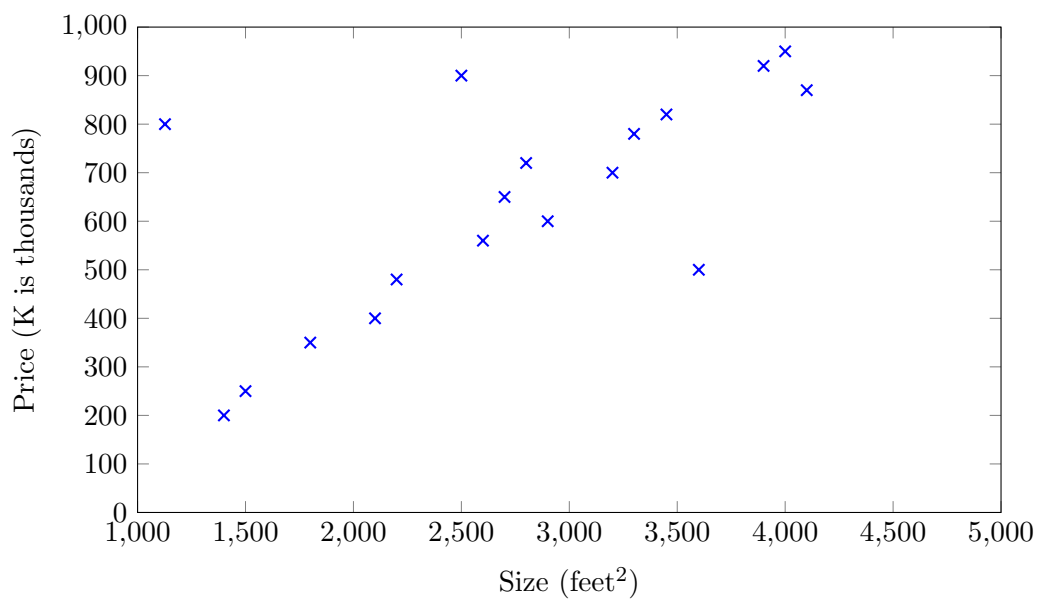
- Regression here is not necessarily broader than the univariate or multivariate versions in statistics. We will sometimes consider high-dimensional regression problems.
- Classification and regression will be used interchangeably in some contexts.
- A regression model that predicts continuous house prices may not perform equally well in classification tasks.
- Example of classification: Predicting whether a house price is above or below \$1,000,000.
- Errors near the classification boundary are critical and should be handled carefully.

Let's examine an example and delve into more details.

Size (feet ²)	Price (K is thousands)
2100	\$400K
2500	\$900K
1127	\$800K
$\vdots$	$\vdots$



Training Set
 $h : \text{feet}^2 \longrightarrow \text{price}$



Given data like this, how can we learn to predict the prices of houses, as a function of the size of their living areas, that are not in the training data?

VII. Representation of h

We aim to represent h as a function mapping one set to another. Since there are infinite possible functions, we need some restrictions to define a proper representation. The approach: **parameterization** of functions.

To establish notation for future use, we'll use:

- $x^{(i)}$: to denote the input variables (e.g., size in this example), also called input **features**
- $y^{(i)}$: to denote the **output** or **target** variable that we are trying to predict (e.g., price).
- A pair (x, y) is called a **training example** and $(x^{(i)}, y^{(i)})$ is the i^{th} example in $i = 1 \dots n$.
- The list of n training examples $\{(x^{(i)}, y^{(i)}); i = 1 \dots n\}$ is called a **training set**.
- The superscript (i) in the notation is simply an index into the training set and has nothing to do with exponentiation.

When the target variable we're trying to predict is continuous, such as in our housing example, we call the learning problem a **regression** problem. When y can take on only a small number of discrete values (such as if, given the living area, we wanted to predict if a dwelling is a house or an apartment), we call it a **classification** problem.

Affine (Linear) Functions

We focus on **affine (or linear) functions** for representation. In **machine learning**, these are often called linear functions, even though they are technically affine. An **affine function** is simply a linear function plus a constant term.

$$h(x) = \theta_0 + \theta_1 x \quad (\text{affine function}). \quad (2)$$

Training Set and Data Representation

Maybe we also have other information like the bedrooms, Lot size, lets break down our **training set**:

	Size (feet ²)	Bedrooms	Lot size	Price (K is thousands)
$x^{(1)}$	2100	4	45k	\$400K
$x^{(2)}$	2500	3	30k	\$900K
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

- Notation:
 - $x^{(i)}$ represents the i -th training example.
 - Each feature is denoted as $x_j^{(i)}$, where j is the feature index.

	Size (feet ²)	Bedrooms	Lot size	Price (K is thousands)
$x^{(1)}$	2100	4 $x_2^{(1)}$	45k	400K
$x^{(2)}$	2500	3	30k	900K
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

If we had all these extra values, we should somehow be able to get a better price estimate

Here, the $\mathbf{x}$'s are three-dimensional vectors ($x \in \mathbb{R}^3$):

- $x_1^{(i)}$ represents the **size** of the i -th house in the training set.
- $x_2^{(i)}$ represents the **number of bedrooms** in that house.
- $x_3^{(i)}$ represents the **lot size** in that house.

Mathematical Representation of Hypothesis h

To perform supervised learning, we must decide how to represent hypotheses $\mathbf{h}$ in a computer. As an initial choice, we approximate y as a linear function of $\mathbf{x}$:

$$h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \theta_3 x_3^{(i)} = \sum_{j=0}^d \theta_j x_j^{(i)}$$

where θ_i are the **parameters** (also called **weights**) that define the linear function mapping from X to Y . Picking those parameters is now the job of the learning function, which we'll discuss. Both θ and $x^{(i)}$ are vectors and d is the number of input features (excluding x_0), also referred to as the dimensionality of the input vector.

To simplify notation, we introduce the convention $x_0 = 1$. x_0 is sometimes referred to as the bias or intercept term. It enables us to express the hypothesis in vector notation.

$$h(x^{(i)}) = \sum_{j=0}^d \theta_j x_j^{(i)} = \theta^T x^{(i)}$$

Where θ and $x^{(i)}$ are both vectors of length d :

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{bmatrix} \quad x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix}$$

When there is no risk of confusion, we drop the θ subscript and simply write:

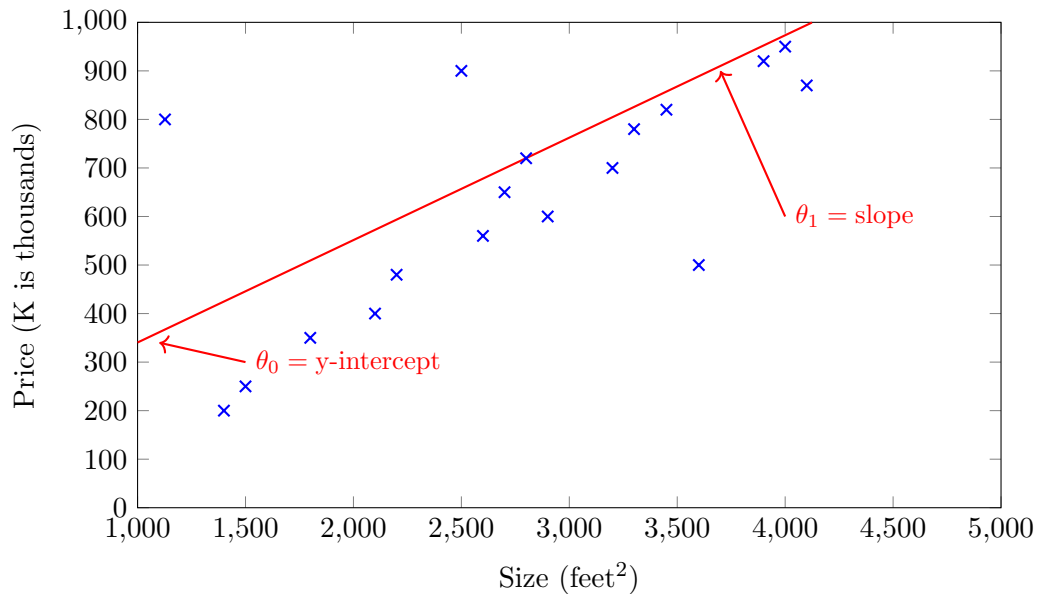
$$h(x) = \theta^T x$$

Additionally, an x without the superscript (i) is frequently used to represent a single input vector.

Given a training set, define the **design matrix** $X \in \mathbb{R}^{(n \times d+1)}$ to be the $n \times d$ matrix (actually $n \times d + 1$ if we include the intercept term) that contains the training examples' input values in its rows:

$$X = \begin{bmatrix} - (x^{(1)})^T - \\ - (x^{(2)})^T - \\ \vdots \\ - (x^{(n)})^T - \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$$

Looking at a concrete example, using the data from above, the learning algorithm will pick $h(x) = \theta_0 + \theta_1 x$, which will be a line. θ_0 is the y-intercept and θ_1 is the slope of the line. To predict the value of a new house, we simply plug the sq. ft into x and compute $h(x)$.



VIII. Cost function

Given a training set, how do we determine the parameters θ ? A reasonable approach is to make $h(x)$ as close as possible to y for the training examples. In other words, we want to pick θ such that $h_{\theta}(x) \approx y$.

To formalize this, we define a function that measures how close $h(x^{(i)})$ is to $y^{(i)}$ for all training examples. This function is called the **cost function** or **loss function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

This is the familiar **least-squares cost function**, which gives rise to the **ordinary least squares** linear regression model. The goal is to minimize J over θ . (The $\frac{1}{2}$ in front of the summation is for convention because that will cancel with the 2 exponent when we take the derivative.)

Even if you have not encountered this function before, we will see later that it is a special case of a much broader class of optimization algorithms.

IX. Common questions

Why Set $x_0 = 1$? Why not choose another constant, like 5 or 0?

- The choice of $x_0 = 1$ ensures that:
 - The bias term θ_0 is included.
 - The scaling remains consistent.
- If another value (e.g., 5) was used, θ_0 would adjust accordingly.

*These functions $h_{\theta}(x)$ are **affine** because:*

- They include a decision variable θ (parameter).
- Each feature x_j is multiplied by a corresponding weight θ_j .

X. Summary

Supervised learning helps extract patterns from labeled data to make accurate predictions for future inputs.

- **Supervised Learning:** The model learns from labeled data, mapping inputs to outputs.
- **Types of Tasks:**

- **Classification:** Predicts categorical labels (e.g., spam detection).
- **Regression:** Predicts continuous values (e.g., house price estimation).
- **Training and Generalization:** A well-trained model should perform well on unseen test data.
- **Model Representation:**
 - A simple linear model:

$$h(x) = \theta_0 + \theta_1 x$$

- Features can be expanded (e.g., polynomial regression) to improve accuracy.